

Assignment 1

Introduction to Web Development

Question 1: Frontend, backend, and full-stack development

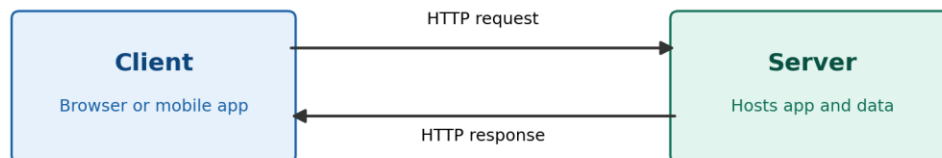
Frontend development is everything the user sees and interacts with directly in the browser — layout, colors, buttons, forms, and animations. It is built using HTML, CSS, and JavaScript, often with frameworks such as React or Vue. Example: the menu, cart page, and checkout screen of an e-commerce site like Amazon.

Backend development is the server-side logic that the user never sees directly — handling requests, running business logic, talking to databases, and managing authentication. It is built with languages such as Node.js, Python, Java, or PHP. Example: when a customer clicks “Place Order,” the backend checks stock, calculates tax, and processes payment.

Full-stack development means working on both the frontend and backend of an application. Example: a single developer building a personal blog from scratch — designing the pages and building the database that stores the posts.

Question 2: Client-server model diagram

The diagram below shows the basic client-server model used in web architecture.



The client (browser or app) sends an HTTP request to the server. The server processes the request and sends back an HTTP response, which the client then displays.

Question 3: How a browser requests and displays a web page

- **URL entry** — the user types a URL or clicks a link.
- **DNS lookup** — the browser asks a DNS server to translate the domain name into an IP address.
- **TCP/TLS connection** — the browser opens a secure connection to the server at that IP address.
- **HTTP request** — the browser sends a GET request asking for the page.
- **Server processing** — the server runs backend logic, possibly querying a database, and prepares an HTML response.
- **Response received** — the server sends back HTML, CSS, JavaScript, and other assets.
- **Rendering** — the browser's rendering engine parses the HTML into a DOM, applies CSS, executes JavaScript, and paints the final page on screen.

Question 4: Tools required to set up a web development environment

- **Code editor (e.g. VS Code)** — used to write HTML, CSS, and JavaScript code.
- **Web browser (e.g. Chrome)** — used to run and test pages, with built-in DevTools for debugging.
- **Version control (Git)** — tracks changes to code and enables collaboration.
- **Node.js and npm** — runs JavaScript outside the browser and manages packages/libraries.
- **Terminal / command line** — used to run commands, servers, and build tools.
- **Live Server extension** — automatically refreshes the browser as the code is edited.
- **Build tools (e.g. Vite, Webpack)** — bundle and optimize project files for production.
- **Database client (e.g. MySQL Workbench, MongoDB Compass)** — used to view and manage data once a backend is involved.

Question 5: Web servers

A web server is software (and often the hardware it runs on) that stores website files and delivers them to clients over HTTP/HTTPS when requested.

Commonly used web servers

- **Apache HTTP Server** — one of the oldest and most widely used, highly configurable.
- **Nginx** — known for speed and handling high traffic; often used as a reverse proxy.
- **Microsoft IIS** — used for Windows-based hosting, especially ASP.NET applications.
- **Node.js (with Express)** — lets developers build a custom server using JavaScript.
- **LiteSpeed** — a high-performance alternative popular with hosting providers.

Question 6: Roles in a project

- **Frontend developer** — builds the user interface, ensures responsiveness, handles interactivity, and connects to backend APIs to display data.
- **Backend developer** — writes server logic, creates APIs, handles authentication and business rules, and connects the application to the database.
- **Database administrator (DBA)** — designs the database schema, manages performance, and ensures data backups, security, and integrity.

Question 7: Installing and configuring VS Code

Steps to install VS Code and configure it for HTML, CSS, and JavaScript development:

1. **Download VS Code** — go to code.visualstudio.com and download the installer for your operating system.
2. **Install it** — run the installer, accepting the default options (on Windows, also check “Add to PATH”).
3. **Install the Live Server extension** — open the Extensions panel (Ctrl+Shift+X), search for “Live Server” by Ritwick Dey, and install it.
4. **Install Prettier and an HTML/CSS/JS pack** — install “Prettier — Code formatter” and optionally “HTML CSS Support” for autocomplete.
5. **Create a test project** — make a folder with index.html, style.css, and script.js, then open it in VS Code.
6. **Take a screenshot** — with VS Code open and the Live Server preview running, capture the screen using your OS's screenshot tool (Windows: Win+Shift+S, Mac: Cmd+Shift+4).

[Insert your own screenshot of the VS Code setup here before submitting.]

Question 8: Static vs dynamic websites

Static websites serve fixed HTML content — the page looks the same for every visitor and only changes when a developer edits the code. Example: a simple personal portfolio or résumé website.

Dynamic websites generate content on the fly, often based on user input or a database. Example: Facebook, where each user's feed is built in real time from their friends' posts and activity.

Aspect	Static website	Dynamic website
Content	Fixed	Changes per user / request
Speed	Very fast	Slightly slower (server processing)
Backend needed	No	Yes
Example	Personal résumé site	Amazon, Facebook, Gmail

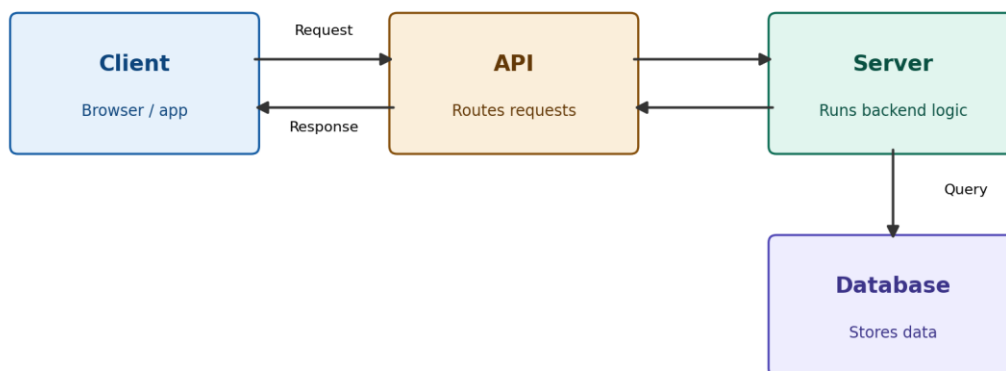
Question 9: Five web browsers and their rendering engines

- **Google Chrome** — uses Blink (a fork of WebKit).
- **Mozilla Firefox** — uses Gecko.
- **Safari** — uses WebKit.
- **Microsoft Edge** — uses Blink (switched from its own EdgeHTML engine in 2020).
- **Opera** — also uses Blink.

The rendering engine parses HTML and CSS and paints the page. Engines differ in how strictly they follow web standards, the JavaScript engine they pair with (Chrome uses V8, Firefox uses SpiderMonkey), animation performance, and support for newer CSS/JS features — which is why the same website can occasionally look or behave slightly differently across browsers.

Question 10: Basic web architecture diagram

The diagram below shows the flow between client, API, server, and database.



The client sends a request through an API layer, which forwards it to the server. The server runs the business logic and queries the database if data is needed, then sends the result back up through the API to the client.